simulator

October 26, 2026


```
This file is part of CasADi.

CasADi -- A symbolic framework for dynamic optimization.
Copyright (C) 2010-2023 Joel Andersson, Joris Gillis, Moritz Diehl,
                        KU Leuven. All rights reserved.
Copyright (C) 2011-2014 Greg Horn

CasADi is free software; you can redistribute it and/or
modify it under the terms of the GNU Lesser General Public
License as published by the Free Software Foundation; either
version 3 of the License, or (at your option) any later version.

CasADi is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the GNU
Lesser General Public License for more details.

You should have received a copy of the GNU Lesser General Public
License along with CasADi; if not, write to the Free Software
Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA  02110-1301  USA
```


1 Simulator

[1]:
```
import casadi as ca
from numpy import *
from pylab import *
```

We will investigate the working of Simulator with the help of the parametrically exited Duffing equation:

$\ddot{u} + \dot{u} - \epsilon(2\mu\dot{u} + \alpha u^3 + 2ku\cos(\Omega t))$ with $\Omega = 2 + \epsilon\sigma$.

[2]:
```
t = ca.SX.sym('t')
```

[3]:
```
u = ca.SX.sym('u')
v = ca.SX.sym('v')
states = ca.vertcat(u,v)
```

[4]:
```
eps   = ca.SX.sym('eps')
mu    = ca.SX.sym('mu')
alpha = ca.SX.sym('alpha')
k     = ca.SX.sym('k')
sigma = ca.SX.sym('sigma')
Omega = 2 + eps*sigma
```

[5]:
```
params = ca.vertcat(eps,mu,alpha,k,sigma)
rhs    = ca.vertcat(v,-u-eps*(2*mu*v+alpha*u**3+2*k*u*cos(Omega*t)))
```

```
/casadi_python/casadi/casadi.py:913: FutureWarning:
casadi: a numpy function was called on a casadi value (issue #2959).
This used legacy casadi 3.7.2 behaviour, where the result type follows
the input:
  - a numeric input (DM)       -> a plain numpy array
  - a symbolic input (SX / MX) -> a casadi value, or an error if the
                                  function has no casadi equivalent

For a shape-correct, type-preserving result, opt in to the
casadi-aware numpy support:

    try:
        ca.GlobalOptions.setNumpyMode(1)
    except AttributeError:
        pass

Or use setNumpyMode(-1) to keep legacy behaviour silently.
Shown once; run python with -W always to see every occurrence.

  _w.warn(_NUMPY_LEGACY_NOTICE, FutureWarning)
```

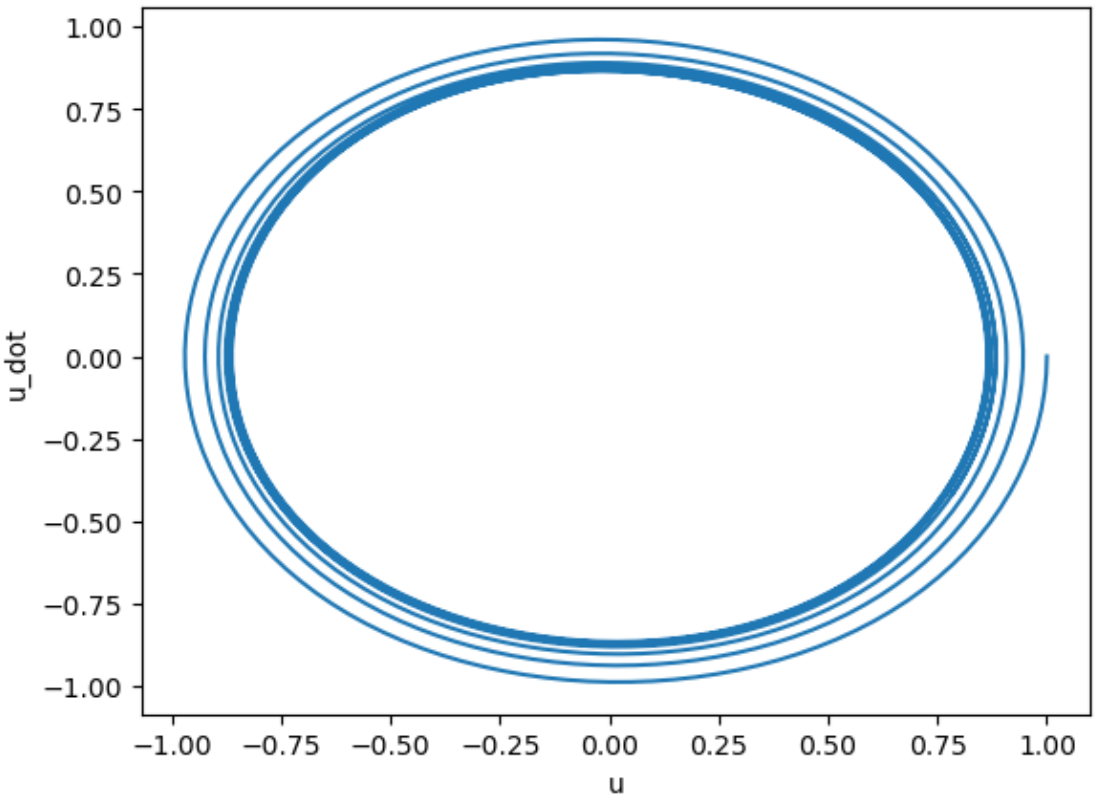
We will simulate over 50 seconds, 1000 timesteps.

[6]:
```
dae={'x':states, 'p':params, 't':t, 'ode':rhs}
ts = linspace(0, 50, 1000)
integrator = ca.integrator('integrator', 'cvodes', dae, 0, ts)
```

[7]:
```
sol = integrator(x0=[1,0], p=[0.1,0.1,0.1,0.3,0.1])
```

Plot the solution

[8]:
```
plot(array(sol['xf'])[0,:], array(sol['xf'])[1,:])
xlabel('u')
ylabel('u_dot')
show()
```


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
u_dot
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
u